

Anexo E • Pruebas unitarias del HomeViewModel

El siguiente código corresponde a las tres pruebas preparadas para ejecutarse sin emulador. **La salida real de flutter test debe adjuntarse después de ejecutar el proyecto en un entorno con Flutter instalado.**

```
import 'package:flutter_test/flutter_test.dart';
import 'package:gameon/features/home/domain/entities/institucion_deportiva.dart';
import
'package:gameon/features/home/domain/repositories/instituciones_repository.dart';
import
'package:gameon/features/home/domain/usecases/obtener_instituciones_activas.dart';
import 'package:gameon/features/home/presentation/states/ui_state.dart';
import 'package:gameon/features/home/viewmodels/home_viewmodel.dart';
```

```
class RepositorioFalso implements InstitucionesRepository {
  final List<InstitucionDeportiva>? respuesta;
  final Object? error;
```

```
  RepositorioFalso({this.respuesta, this.error});
```

```
  @override
```

```
  Future<List<InstitucionDeportiva>> obtenerInstitucionesActivas() async {
    if (error != null) throw error!;
    return respuesta ?? const [];
  }
```

```
}
```

```
const institucionDePrueba = InstitucionDeportiva(
  id: 1,
  usuarioInstalacionId: 10,
  nombre: 'Complejo Deportivo Tacna',
  direccion: 'Av. Principal 123',
  latitud: -18.0066,
  longitud: -70.2463,
  tarifa: 40,
  calificacion: 4.5,
  telefono: '999999999',
  email: 'contacto@example.com',
  estado: 1,
);
```

```

void main() {
  test('emite Loading y luego Success cuando el repositorio responde', 0 async {
    final vm = HomeViewModel(
      obtenerInstitucionesActivas: ObtenerInstitucionesActivas(
        RepositorioFalso(respuesta: const [institucionDePrueba]),
      ),
    );

    expect(vm.estado, isA<Loading<List<InstitucionDeportiva>>>());
    await vm.cargar();

    expect(vm.estado, isA<Success<List<InstitucionDeportiva>>>());
    final estado = vm.estado as Success<List<InstitucionDeportiva>>;
    expect(estado.data, hasLength(1));
    expect(estado.data.first.nombre, 'Complejo Deportivo Tacna');
  });

  test('emite Empty cuando el repositorio responde sin elementos', 0 async {
    final vm = HomeViewModel(
      obtenerInstitucionesActivas: ObtenerInstitucionesActivas(
        RepositorioFalso(respuesta: const []),
      ),
    );

    await vm.cargar();

    expect(vm.estado, isA<Empty<List<InstitucionDeportiva>>>());
  });

  test('emite Error con acción de reintento cuando el repositorio falla', 0 async {
    final vm = HomeViewModel(
      obtenerInstitucionesActivas: ObtenerInstitucionesActivas(
        RepositorioFalso(error: Exception('sin conexión')),
      ),
    );

    await vm.cargar();

    expect(vm.estado, isA<ErrorState<List<InstitucionDeportiva>>>());
    final estado = vm.estado as ErrorState<List<InstitucionDeportiva>>;
    expect(estado.retry, isNotNull);
    expect(estado.message, isEmpty);
  });
}

```

```
});  
}
```